# Operating system II

*Chapitre II*: Process Synchronization

K.Belarbi

Université des sciences et de technologie Med Boudiaf Oran
Faculté MI Département Informatique
06 Janvier 2025

20 mars 2025

# Process Synchronization

## Objectifs du chapitre

- Présenter le problème de critical section, dont les solutions peuvent être utilisées pour assurer la cohérence des données partagées.
- Présenter les solutions logicielles et matérielles du problème de critical section.
- Examiner plusieurs problèmes classiques du process synchronization
- Explorer plusieurs outils utilisés pour résoudre process synchronization.

# Process Synchronization

## Définitions

La mémoire est partagée entre plusieurs process, il y a donc des risques de comportements bizarres.
Situation de compétition ou de coopération entre process

- **Compétition :** plusieurs process veulent accéder à la même ressource *(par exemple, modifier le même fichier ou la même variable)*
- **Coopération :** plusieurs process interagissent pour mener à bien une tâche – ils doivent communiquer régulièrement pour échanger des informations et déterminer l'avancement global
- Les deux situations nécessitent de synchroniser les process

# Process Synchronization

## Exemple - *Compte Bancaire-*

Deux opérations sur le même compte exécutées en concurrence :

**Concurrence sans problème**

| tâche 1 (compte += 1000) | tâche 2 (compte -= 500) | ressources compte | r1 | r2 |
|---|---|---|---|---|
| charger (r1, compte) | | 2000 | **2000** | 0 |
| charger (r2, 1000) | | 2000 | 2000 | **1000** |
| ajouter (r1, r2) | | 2000 | **3000** | 1000 |
| écrire (r1, compte) | | **3000** | 3000 | 1000 |
| *sauvegarde contexte* | | | 3000 | 1000 |
| | *restauration contexte* | | **0** | **0** |
| | charger (r1, compte) | 3000 | **3000** | 0 |
| | charger (r2, 500) | 3000 | 3000 | **500** |
| | soustraire (r1, r2) | 3000 | **2500** | 500 |
| | écrire (r1, compte) | **2500** | 2500 | 500 |

# Process Synchronization

## Exemple - Compte Bancaire-

Deux opérations sur le même compte exécutées en concurrence :Concurrence avec problème

| tâche 1 (compte += 1000) | tâche 2 (compte -= 500) | ressources compte | r1 | r2 |
|---|---|---|---|---|
| charger (r1, compte) | | 2000 | 2000 | 0 |
| charger (r2, 1000) | | 2000 | 2000 | 1000 |
| ajouter (r1, r2) | | 2000 | 3000 | 1000 |
| *commutation de contexte →* | | | 0 | 0 |
| | charger (r1, compte) | 2000 | 2000 | 0 |
| | charger (r2, 500) | 2000 | 2000 | 500 |
| | soustraire (r1, r2) | 2000 | 1500 | 500 |
| | écrire (r1, compte) | 1500 | 1500 | 500 |
| *← commutation de contexte* | | | 3000 | 1000 |
| écrire (r1, compte) | | 3000 | 3000 | 1000 |

- Le résultat dépend de l'entrelacement des actions, donc de l'ordonnancement des process. Il y a donc un besoin de contrôler le déroulement d'un process.

- Il faut empêcher l'utilisation simultanée en lecture et écriture de la variable commune ***Compte***.

# The Critical-section problem

## Concepts fondamentaux

### Ressource critique

Une *ressource* est dite **ressource critique** lorsque des accès concurrents à cette ressources peuvent résulter dans un état incohérent.

### Race condition

On parle aussi de **situation de compétition**(race condition) pour décrire une situation dont l'issue dépend de l'ordre dans lequel les opérations sont effectuées.

### Section atomique

Une section de programme est dite **atomique** lorsqu'elle ne peut pas être interrompue par un autre process manipulant les mêmes ressources critiques. C'est donc une atomicité relative à la ressource.

# The Critical-section problem

## Concepts fondamentaux

### Critical-section

Critical-section est une section de programme manipulant une ressource critique.

### Mutual exclusion

Un mécanisme d'exclusion mutuelle sert à assurer l'atomicité des sections critiques relatives à une ressource critique.
Une ressource est en exclusion mutuelle si seul un processus peut utiliser la ressource à un instant donné.

# The Critical-section problem

## Caractéristique

- Chaque process possède un segment de code, appelé **critical-section**
  - Un process modifie des variables communes, actualiser une table, écrire un fichier, etc
- Quand un process exécute sa *critical-section*, aucun autre process n'est autorisé à exécuter sa *critical-section* (Un seul process dans critical-section) ;
- L'exécution des *criticals-sections* est mutuellement exclusive dans le temps ;

Le **problème** de criticals sections est de concevoir un protocole que les process puissent utiliser pour coopérer.

# Critical Section Problem

## Caractéristique

- ***La tolérance aux défaillances :*** si le processus en SC est détruit ou se termine anormalement, il ne faut pas qu'il bloque tout le système ;
- ***La symétrie*** : Les protocoles d'E/S en SC doivent être identiques pour tous les processus et indépendants de leur nombre.
- Aucune hypothèse ne doit être posée sur les vitesses relatives des processus ni sur le nombre de processeurs.
- L'exclusion Mutuelle n'est pas garantie si :
  1. un processus peut entrer en SC alors qu'un autre s'y trouve déjà ;
  2. un processus désirant entrer en SC ne peut pas y entrer alors qu'il n'y a aucun processus en SC ;
  3. un processus désirant entrer en SC n'y entrera jamais car il sera jamais sélectionné lorsqu'il est en concurrence avec d'autres processus.

# Critical Section Problem

## Caractéristique

- chaque process doit demandé la permission d'entrer à sa Critical Section.
  - **Entry section** est la section code qui implémente cette requête ;
  - Critical Section doit être suivie par **Exit section** ;
  - Le code restant est **remainder section** ;

```
do {
    [entry section]
        critical section
    [exit section]
        remainder section
} while (true);
```

Figure – General structure of a typical process Pi

# Critical Section Problem

## Exemple Banque

| P1 | P2 |
|---|---|
| **entry_Critical_section(....)**; | **entry_Critical_section( .....)**; |
| /* début la section critique */ | /* début la section critique */ |
| int tmp = compte; | int tmp = compte; |
| tmp = tmp + 1000; | tmp = tmp -500; |
| compte = tmp; | compte = tmp; |
| /* fin de la section critique */ | /* fin de la section critique */ |
| **Exit_Critical_section(.....)** | **Exit_Critical_section(....);** |

**? ?**

Les codes de fonctions entry_Critical_section(....); et Exit_Critical_section(.....);

# Critical-section problem

## Solutions

Une solution au critical-section problem, doit satisfaire aux trois besoins suivants :

- **Mutual Exclusion :** si le process $P_i$ exécute sa critical-section, aucun autre process ne peut exécuter sa critical-section ;
  Idée : verrouiller l'accès à une section critique déjà occupée
- **Progress :** si aucun process n'exécute sa critical section et si certains process désirent entrer dans les leurs, alors seulement les process qui ne se trouvent pas dans leurs remainder section peuvent décider qui rentrera dans critical-section (la sélection ne peut pas être reportée indéfiniment).
- **bounded waiting :** limiter le nombre de permissions à d'autres process a accédé à critical section ;

# Critical section problem

## Solutions Critical-section

- Solutions logicielles pour le problème de critical-section
  - Solutions algorithmiques
    – Exemples : Solution de Peterson, Solution de Dekker (Fiche TD2 )
    – Limitées à deux processus
    – Pas de garantie de bonne fonctionnements sur toutes les architectures
- Solutions matérielles
  - Masquage des interruptions
    - Pas faisable sur des architectures multiprocesseurs
    - Peut perturber le bon fonctionnement du système (mise à jour de l'horloge)
  - Instructions spéciales (supportées par le matériel)
    - Test and set (difficile dans un multiprocesseur)
    - Permutation (swap) atomique
    - Complexes à utiliser par les programmeurs

# Critical-section solution

## Le verrou -attente active-

**Verrou**

Un mécanisme proposé pour permettre de résoudre les concurrences d'accès à une ressource est **le mécanisme de verrou**.

On utilise un **booléen partagé** comme Verrou
**Un verrou** est un objet système à **deux états** (libre(unlocked)/occupé(locked)) sur lequel deux opérations sont définies.

- Lock(v) permet au process d'acquérir le verrou v s'il est libre. S'il n'est pas disponible, le process est bloqué en attente de la ressource.
- Unlock (v) permet au process de libérer le verrou v qu'il possédait.

Si un ou plusieurs process étaient en attente de ce verrou, un seul de ces process est réactivé et reçoit le verrou.
invoquer unlock() réveille un process suspendu (s'il y en a) attention : ordre de réveil non spécifié

# Critical-section solution

## Verrou -attente active-

**V_IMP : verrou; -- verrou libre**

**Processus 1**

**PRINT()**

**Verrouiller (V_IMP)**

**V_IMP libre**

**V_IMP occupé par processus1**

***IMPRESSION***

**Processus 2**

**PRINT()**

**Verrouiller (V_IMP)**

**V_IMP Occupé par processus1**

***Processus 2 bloqué***

**Deverrouiller (V_IMP)**

**Réveil de processus 2**

**Verrouiller (V_IMP)**

***IMPRESSION***

# Critical-section solution

## API POSIX (Threads)

### API POSIX : Mutex locks (Threads)

```
1 #include <pthread.h>
2
3
4 /* opaque typedefs */ pthread_mutex_t, pthread_mutexattr_t;
5
6 // create a new mutex lock}
7
8  int pthread_mutex_init(pthread_mutex_t *mutex,
9      pthread_mutexattr_t *mutexattr);
10
11  int pthread_mutex_lock(pthread_mutex_t *mutex);
12
13  int pthread_mutex_unlock(pthread_mutex_t *mutex);
14
15 // attempt to lock a mutex without blocking
16
17  int pthread_mutex_trylock(pthread_mutex_t *mutex);
18
```

# Critical-section solution

## attente active

**Défauts de l'attente active** Le processus qui attend consomme du temps processeur (d'où le nom d'attente active).
**NB :** on peut améliorer cela en mettant un *sleep* ou un *yield* dans la boucle.
Cette approche n'assure par l'exclusion !
verrou est lui même une ressource critique.
- **Bonne solution**
On associe à la ressource un jeton, que les process peuvent prendre et reposer Seul le process possédant le jeton devrait manipuler la ressource Donc, si un process souhaite manipuler la ressource et que le jeton est pris, il doit d'abord attendre que le jeton redevienne disponible
*Utilisation d'un sémaphore initialisé à 1*

# Process Synchronization

## Résumé

### Exclusion mutuelle

**Notion de "race condition"** :

- plusieurs accès concurrents à une même variable
- accès non atomique données incohérentes

**Critical section**

- morceau de code qu'on veut rendre atomique
- exécution nécessairement en exclusion mutuelle

**Solution :** utiliser un mutex lock

```
lock(V) ;
/* section critique */
unlock(V) ;
```

### Résumé des problèmes

- Attentes actives à consommation du temps CPU.

• Masquage des interruptions à dangereuse pour le système infinie. • SLEEP et WAKEUP à mauvaise synchronisation des signaux (rendez-vous manqués) à blocage.

# Process Synchronization

## Sémaphore

- Les solutions matérielles et logicielles présentées dans les sections précédentes sont difficiles à mettre en œuvre pour des problèmes de synchronisation complexes.
- Le système d'exploitation offre un outil de synchronisation appelé **sémaphore**,
- Les sémaphores ont été introduits par ***Edsger Dijkstra***, informaticien hollandais , en 1965.
- Un *sémaphore* **S** est constitué de deux champs : d'un compteur (une variable)protéger et d'une file d'attente **F** *(Process bloqués)*

# Sémaphore

## Implémentation

On peut accéder au sémaphores à l'aide des Primitives suivants :

- ***Initialisation* I(S,x)** : qui permet de créer le sémaphore et de lui attribuer une valeur initiale (x) positive ou null ;
- ***Manipulation* :** par deux opérations standards atomiques wait et signal ;
- Une fonction permettant de détruire un sémaphore et de libérer les ressources qui lui sont associées.

# Sémaphore

## Implémentation WAIT

- **wait**, P(Dijkstra(**P**roberen, tester))(down(algol68), sleep)
  **wait(S)** : demande d'autorisation, qui est utilisée pour modifier la valeur du sémaphore.
  - Si S>0, S- - *(décrémenter)* (en utilisant un wakeup stockée) et poursuit son activité
  - Si S==0 ,le process est placé en sommeil(file d'attente) sans que down ne se termine
  - Si F(S) vide, S++, Sinon, un des process en attente (choix aléatoire) est libéré et passe à l'état Ready.

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

# Sémaphore

## Implémentation SIGNAL

- Signal, V(Dijkstra(**V**erhogen,incrémenter),(up(algol68), wakeup)
  **signal(S)** : fin d'utilisation (Libérer)
  - Si F(S) non vide Sinon, un des process en attente (choix aléatoire) est libéré et passe à l'état Ready et S==0 sinon S++
  - Si S>0, S– d'une unité et la fonction réussit.
  - Si s==0, le process est bloqué jusqu'à ce qu'un autre process le débloque en appelant la fonction post.
- Quant un process doit attendre un sémaphore, il est ajouté à la file de process. Une opération Signal supprime un process de la file des process en attente et réveille ce process.
- L'exécution d'une opération wait(s) ou signal(s) se fait sans interaction possible (de façon atomique).

```
signal(S) {
    S++;
}
```

# Type de sémaphore

## Sémaphore binaires

- Un sémaphore binaire est un sémaphore ayant une valeur entière entre 0 et 1 ;
- Un sémaphore binaire est un sémaphore qui est **initialisé avec la valeur 1**. Ceci a pour effet de contrôler l'accès une ressource unique.
- Le sémaphore binaire permet l'exclusion mutuelle (mutex) : Une ressource est en exclusion mutuelle si seul un processus peut utiliser la ressource à un instant donné.
  Un sémaphore qui est initialisé avec la valeur 0 est un sémaphore bloquant.
- Plus simple à implémenter qu'un sémaphore de comptage

# Type de sémaphore

## Sémaphore de comptage

Un sémaphore de comptage la valeur peut prendre plus de deux valeurs positives possibles.
Il est utile pour allouer une ressource parmi plusieurs exemplaires identiques : la valeur est initialisée avec le nombre de ressources.

# Sémaphore

## Sémaphore (Utilisation)

### Problème de la section critique

n processus partagent un sémaphore,
Les sémaphores binaires permettent d'assurer l'exclusion mutuelle :
Semaphore mutex = 1 ;
Principe :

- Sémaphore mutex (à cause de mutual exclusion)initialisé a 1 ;
- Primitive wait en début de la section critique ;
- Primitive signal en fin de la section critique ;

Implémentation de Mutual exclusion avec des semaphores.

**repeat**

wait(mutex) ;

critical section

signal mutex()

section restante

**until** false ;

*Deux Process P1 et P2 exécutent respectivement deux instructions S1 et S2*

| Process P1 | Process P2 |
|---|---|
| *Debut* | *Debut* |
| S1 | S2 |
| *Fin* | *Fin* |

*S2 s'execute après S1*

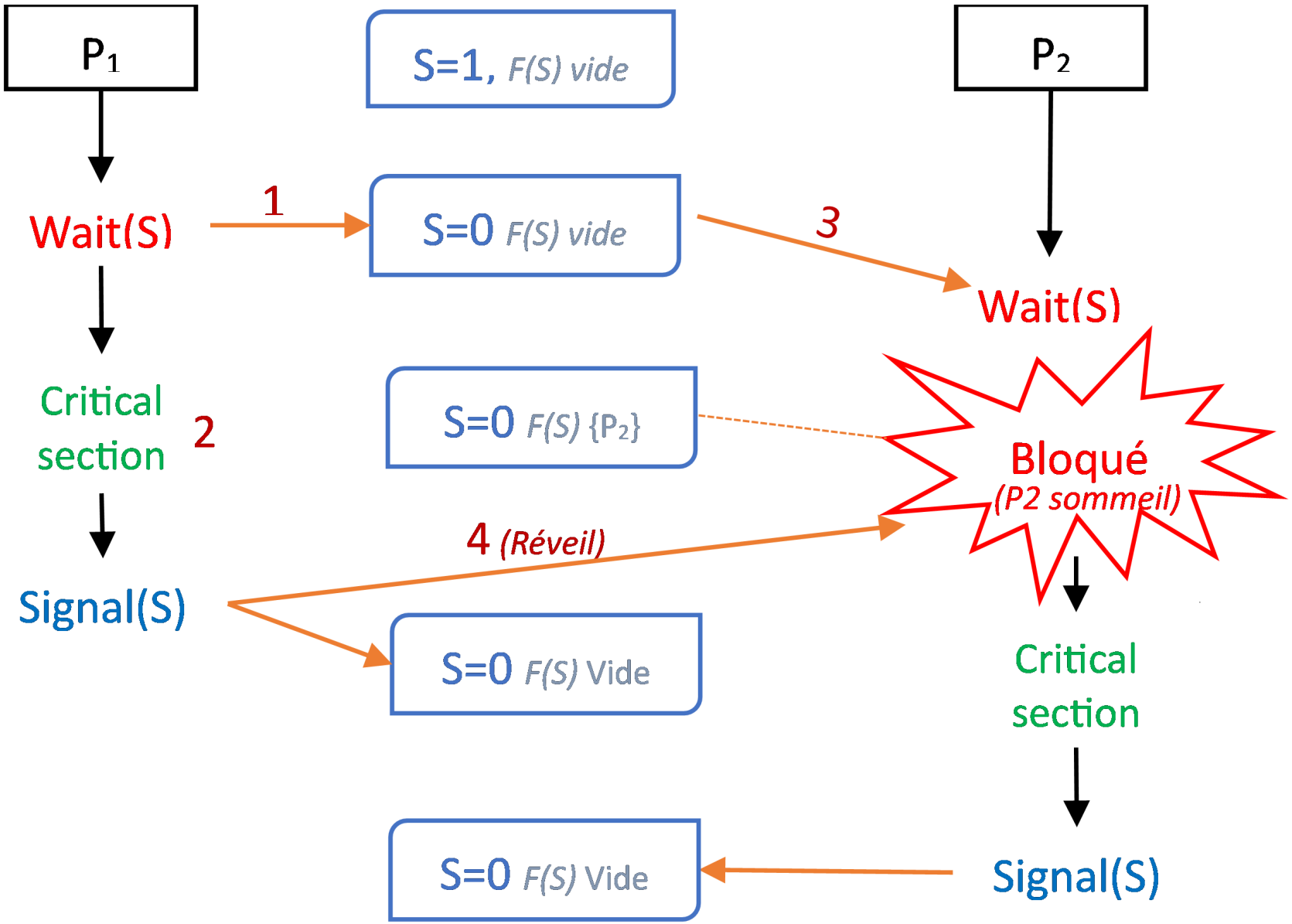
| Process P1 | Process P2 |
|---|---|
| *Debut* | *Debut* |
| S1 ; | Wait(S) ; |
| Signal(S) | S2 |
| *Fin* | *Fin* |

Comme S est initialisé à 0, Process P2 executera S2 seulement une fois que Process P1 aura appelé *signal(S)*, qui est après S1.

**Remarque :** Un sémaphore peut être initialisé à n'importe quelle valeur entière, mais généralement cette valeur est positive ou null.

# Sémaphore

## Exemple



# Sémaphores

## Sémaphores POSIX

Les sémaphores POSIX sont implantés dans la librairie <semaphore.h>
Le type sémaphore est désigné par le mot : sem_t.

```
#include <semaphore.h>

int sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_destroy(sem_t *sem);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
```

# Sémaphores

## Sémaphores POSIX

**Initialisation** :

```
#include <semaphore.h>
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

Une fois l'objet créé, on peut l'initialiser :

*sem* $\mapsto$ pointeur vers le sémaphore à initialiser ;

*pshared* $\mapsto$ drapeau qui précise si le sémaphore est utilisé par des threads (valeur 0pour local et non null pour partagé) ;

*value* $\mapsto$ valeur de départ(initiale) du sémaphore.

le code retour varie entre :

'**0**' si tout s'est bien passé

'**1**' si une erreur survient et *errno* est positionné

# Sémaphores

## Sémaphores POSIX

**Blocage**

Une fois le sémaphore initialisé, on peut demander son blocage avec

```
#include <semaphore.h>
int sem_wait(sem_t *sem) ;
```

sem ↦ pointeur vers le sémaphore à bloquer ;

le code retour varie entre :

'**0**' si tout s'est bien passé

'**-1**' si une erreur survient et *errno* est positionné

```
int sem_wait(semaphore *s)
  {
    s⟶val = s⟶val-1 ;
    if(s⟶val<0)
      {
        // Place this thread in s⟶queue ;
        // This thread is blocked ;
      }
  }
```

# Sémaphores

## Sémaphores POSIX

**Relachement d'un sémaphore**

Une fois la section critique passée, on peut relâcher le sémaphore :

```
#include <semaphore.h>
int sem_post(sem_t *semaphore) ;
```

semaphore⟶ pointeur vers le sémaphore bloqué ;

le code retour varie entre :

'**0**' si tout s'est bien passé

'**-1**' si une erreur survient et errno est positionné

```
int sem_post(semaphore *s)
    {
      s⟶val = s⟶val+1 ;
      if(s⟶val<=0)
        {
          // Remove one thread(T) from s⟶queue ;
          // Mark Thread(T) as ready to run ;
        }
    }
```

**Destruction**

int sem_destroy(sem_t *sem) ;

sem $\longrightarrow$ pointeur vers le sémaphore à détruire ;

le code retour varie entre :

'**0**' si tout s'est bien passé

'**-1**' si une erreur survient et errno est positionné

int sem_trywait( sem_t *sem) : décrémente la valeur du sémaphore, si sa valeur est supérieure à 0 et retourne 0 ; sinon elle retourne une erreur (une valeur différente de 0) => sem_wait non bloquant.

int sem_getvalue(sem_t *sem, int * sval) : retourne dans sval la valeur courante du sémaphore.

Surmonter le besoin de l'attente active :

- Modifier la définition des opérations de sémaphores *Wait* et *signal*.
- Qd un processus exécute wait(), au lieu de l'attente, le processus peut se bloqué lui même
- Opération block place le un process dans une file d'attente associés au sémaphore.
- Le contrôle est transféré au sheduler de l'UC qui sélectionne un autre process (pour l'exécuter)
- un process bloqué devrait être redémarré qd un autre process exécute *signal*.
- process est redémarré avec une opération *wakeup* (changement d'état de attente à prêt)

**Définition 2**

Un sémaphore S est un ensemble de deux variables :

- une valeur entière value
- une file d'attente F de process

**Implémentation**

décrire un sémaphore comme un enregistrement

**type** *semaphore*=**record**
*value : integer ;*
L :list of *process* ;
**end** ;

Les opérations des sémaphores peuvent être défini comme suit :

```
wait(s) : S.value :=S.value-1 ;
                    if S.value<0
                       then begin
                              ajouter ce process à S.L ;
                              block ;
                       end ;

signal(S) : S.value :=S.value+1 ;
                    if S.value≤0
                       then begin
                              enlever le process P à S.L ;
                              wakeup(P) ;
                       end ;
```

La file d'attente d'un sémaphore est gérée :
– FIFO : sémaphore fort (par défaut) ou
– LIFO : sémaphore faible (famine possible).

# Interblocage et famine

Deux process ou plus peuvent attendre indéfiniment un événement qui ne peut être produit que par un process en attente (l'exécution de signal).

Ces process se trouvent dans une situation d'**interblocage**

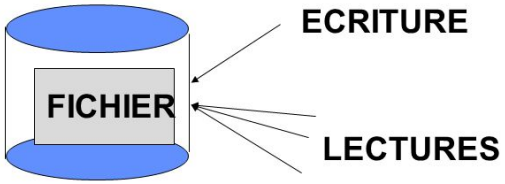
Si nous ajoutons ou supprimons des process de la file dans un ordre LIFO, il peut de produire un blocage indéfini ou la **famine**. Une situation où les process attendent indéfiniment dans le sémaphore.

# Problème classique de synchronisation

## Le problème des lecteurs et rédacteurs

- Ce problème traite de l'accès concurrent en lecture et en écriture à une ressource : *une base de donnée*. *exple* :réservation de billets d'avion
- Plusieurs processus légers (thread) peuvent lire en même temps la ressource (lecteurs), aucun effet défavorable (puisqu'une lecture ne modifie pas le contenu d'une ressource) ;
- Si on veut actualiser (lire et écrire)-rédacteur- : si un rédacteur et d'autres process( lecteurs où rédacteurs) accèdent simultanément à la ressource, le chaos peut s'ensuivre. les lecteurs ne doivent pas lire une information en cours de modification ;
- Les rédacteurs demandent des accès en écriture à la ressource ;
- Les lecteurs demandent des accès en lecture à la ressource.

**Les sémaphores**
**Lecteurs / Rédacteurs**




**• Ecriture seule - Lectures simultanées**

| | |
|---|---|
| ☞ **Un écrivain exclut** | **- les écrivains**<br>**- les lecteurs** |
| ☞ **Un lecteur exclut** | **- les écrivains** |

# Problème classique de synchronisation

## Le problème des lecteurs et rédacteurs

### Le rédacteur (écrivain)

**Un écrivain exclut les autres écrivains et les lecteurs** : un écrivain accède donc toujours seul à la ressource, autrement dit il effectue des accès en mutual exclusion des autres écrivains et des lecteurs (un accès exclusif).

- Le schéma du mutual exclusion (rédacteur) se fait à l'aide d'un *sémaphore* **redact** initialisé à 1.

# Problème classique de synchronisation

## Le problème des lecteurs et rédacteurs

**Le lecteur** Un lecteur exclut les écrivains mais pas les autres lecteurs. Il faut donc :

- Le premier lecteur doit s'assurer qu'il n'y a pas d'accès en écriture en cours ;
- Le dernier lecteur doit réveiller un éventuel écrivain ;
- On doit compter le nombre de lecteurs qui accèdent à la ressource. On utilise pour cela une variable NbL, initialisée à 0. Cette variable NbL va être accédée en concurrence par tous les lecteurs qui vont soit incrémenter cette variable (un lecteur de plus), soit la décrémenter (un lecteur de moins). Pour que le contenu de la variable reste cohérent, il faut que NbL soit accédée en exclusion mutuelle. L'accès à la variable sera donc gardé par un *sémaphore* **MUTEX** initialisé à 1.

# Problème classique de synchronisation

## Le problème des lecteurs et rédacteurs

```
1 int NbL=0;                                        /* nombre de lecteurs */
2 sem_t mutex;                                      /* contrôle l'accès à NbL*/
3 sem_t redact;                                     /* contrôle l'accès à la base de donnée*/
4
5 void *redear(void *arg){
6         while(1){                                 /* boucle sans fin*/
7                 sem_wait(&mutex);                 /* obtient l'accès exclusif à NbL*/
8                 if (NbL == 0) sem_wait(&redact);/* si c'est le premier lecteur ... */
9                 NbL++;                            /* et un lecteur de plus*/
10                sem_post(&mutex);                 /* libère l'accès exclusif à NbL*/
11                // lecture de la base
12                sleep(1);
13                printf("lecteur bd=%d\n", db);
14                // fin de l'accès à la base
15                sem_wait(&mutex);                 /*recupère l'accès exclusif à NbL*/
16                NbL--;                            /*et un lecteur de moins*/
17                if(NbL == 0) sem_post(&redact);  /*si c'est le dernier lecteur ... */
18                sem_post(&mutex);                 /* libère l'accès exclusif à NbL*/
19        }
20 }
21 void *writer(void *arg) {
22        while(1) {                                /*boucle sans fin*/
23                sem_wait(&redact);                /* recupère l'accès exclusif */
24                // modifier les données de la base
25                db++;
26                printf("redacteur bd=%d\n", db);
27                sleep(2);
28                sem_post(&redact);                /*libère l'accès exclusif/
29                }
30 }
31
```

# Problème classique de synchronisation

## Le problème des lecteurs et rédacteurs

- Il existe plusieurs variations du problème des lecteurs et rédacteurs prenant en compte les priorités.
- le $1^{er}$ problème : Aucun lecteur ne devrait attendre parce qu'un rédacteur attend $\Rightarrow$ état de famine : manque de ressource et un rédacteur pourra ne jamais entrer ;
- le second problème : un rédacteur est prêt, il effectue son écriture le plus vite possible, aucun nouveau lecteur ne peut commencer à lire $\Rightarrow$ état de famine

Exercice : *Modifiez le code de manière à empêcher les lecteurs d'accéder à la base si un au moins des rédacteurs est en attente.*

Si la ressource est utilisée par un lecteur :

- Tout écrivain est mis en attente.
- Tout lecteur est accepté s'il n'y a pas d'écrivain en attente.
- Tout lecteur est mis en attente s'il y a un écrivain en attente.

# Problème classique de synchronisation

## Le problème du diner des philosophes

**Le diner des philosophes**

Il s'agit d'un problème classique de synchronisation posé et résolu par Dijkstra [1965] qui modélise des processus qui entre en concurrence pour un accès exclusif à un nombre limité de ressources.

C'est une représentation simple du besoin d'allouer plusieurs ressources parmi plusieurs process sans rencontrer le problème de l'inter-blocage ou de la famine.

TD_Sémaphores_Fiche3

# Problème classique de synchronisation

## Le problème Producteur/consommateur

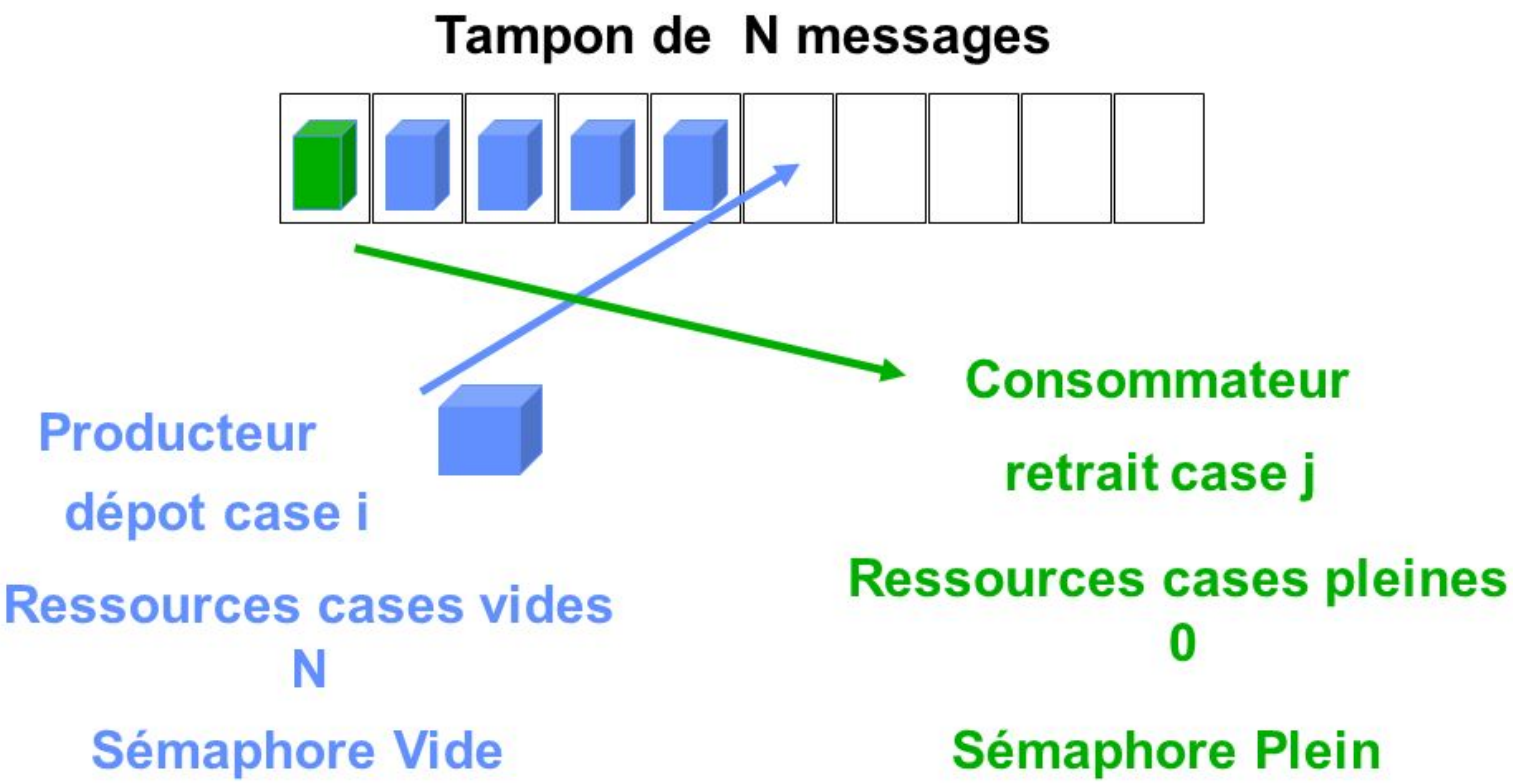
- De nombreuses applications sont construites sur le modèle **client serveur** : Interaction entre un client qui fait une demande et un serveur qui fournit une réponse. Il est largement utilisé dans les architectures réseau.
- *Producteur-consommateur* : Modèle où un producteur génère des données que le consommateur utilise. Il est souvent utilisé pour gérer les flux de données et la synchronisation entre processus dans des systèmes concurrents.
- Les cadences de production et de consommation peuvent varier au cours du temps
- augmenter la probabilité d'exécution parallèle du producteur et du consommateur, ceux-ci communiquent par l'intermédiaire d'un tampon de N cases
- Le principal défi dans le modèle producteur-consommateur est la gestion des ressources partagées (par exemple, une mémoire tampon)
- Un producteur peut produire des données plus rapidement que le consommateur ne peut les traiter, et inversement.

# Problème classique de synchronisation

## Le problème Producteur/consommateur

- Buffer Overflow (dépassement de tampon) : Si le consommateur ne consomme pas les données assez vite, le tampon dans lequel les données sont stockées pourrait déborder.
- Underflow (pénurie de données) : Si le producteur ne produit pas suffisamment de données pour satisfaire le consommateur, ce dernier pourrait se retrouver à attendre sans rien à traiter.

## Les sémaphores Producteur-Consommateur




26

# Conclusion sur les sémaphores

- Les sémaphores sont un excellent mécanisme pour gérer la mise en attente de processus.
- Lorsqu'il faut gérer la mise en attente en combinaison avec la manipulation d'un compteur, les sémaphores sont très bien adaptés.
- L'utilisation des sémaphores binaires indique quelles difficultés apparaissent lorsque l'on combine l'attente implémentée par un sémaphore avec la manipulation d'une autre structure de donnée.
- Il serait intéressant d'avoir une solution systématique pour gérer ce type de problème.

# Les Moniteurs

## Quel est le problème avec les sémaphores ? ? ?

Les sémaphores représentent un énorme progrès par rapport à l'implémentation load/store équivalente, mais ils présentent les inconvénients suivants :
- Il s'agit essentiellement de variables globales partagées.
- il n'y a pas de lien linguistique entre le sémaphore et les données auxquelles le sémaphore contrôle l'accès.
- L'accès aux sémaphores peut provenir de n'importe quel endroit du programme.
- Ils ont deux fonctions : l'exclusion mutuelle et les contraintes d'ordonnancement.
- Il n'y a pas de contrôle ou de garantie d'utilisation correcte.

Solution : **utiliser une primitive de niveau supérieur appelée moniteur.**

- Les moniteurs sont des primitives de synchronisation, introduit par Hoare en 1973, initialement proposées dans les langages objet. Il est utilisé actuellement dans des langages tel que ADA ou Java et implémenté au sein de systèmes d'exploitation.
- Le moniteur se positionne sur une classe et les mutexes sur ses méthodes et sont basés sur des variables condition. elle sont forcement privée et inaccessible à l'extérieur de la classe.
Moniteur = variables globales + sections critiques d'un même problème

- Pour assurer l'exclusion mutuelle, à tout moment un processus au plus est actif dans le moniteur.
- Si un processus est actif dans le moniteur et un autre demande à y accéder (appelle une procédure du moniteur), ce dernier est mis en attente.
- C'est le compilateur qui se charge de cette gestion. Pour ce faire, il rajoute au moniteur le code nécessaire pour assurer l'exclusion mutuelle.
- Cette solution est plus simple que les sémaphores et les compteurs d'événements, puisque le programmeur n'a pas à se préoccuper de contrôler les accès aux sections critiques.
- La majorité des compilateurs utilisés actuellement ne supportent pas les moniteurs (à l'exception de JAVA, C#).

# Les Moniteurs

## Définition formelle

- Un moniteur définit un verrou et zéro ou plusieurs variables de condition pour gérer l'accès concurrent aux données partagées.
  - Le moniteur utilise le verrou pour s'assurer qu'un seul thread est actif dans le moniteur à tout moment.
  - Le verrou permet également l'exclusion mutuelle des données partagées.
  - Les variables de condition permettent aux threads de s'endormir à l'intérieur des sections critiques, en libérant leur verrou en même temps que la mise en veille du thread.
- Opérations de contrôle :
  - Encapsule les données partagées que vous souhaitez protéger.
  - Acquiert le mutex au début.
  - Opère sur les données partagées.
  - Libère temporairement le mutex si l'opération ne peut pas se terminer.
  - Réacquiert le mutex lorsqu'elle peut continuer.
  - Libère le mutex à la fin.

- Il est simple de transformer une classe Java en moniteur :
- Rendre toutes les données privées
- Rendre toutes les méthodes synchronisées (ou au moins celles qui ne sont pas privées)

```
class Queue{
private ... ; // queue data

public void synchronized Add( Object item ) {
    put item on queue ;
}

public Object synchronized Remove() {
     if queue not empty {
        remove item ;
        return item ;
    }
}
```

```
Moniteur Compte
{ int solde = 0;
void Deposer (int montant) // section critique pour le dépôt
{ solde = solde + montant;
}
void retirer (int montant) //section critique pour le retrait
{ if (solde >= montant)
solde = solde - montant;
}
}
```